แผนสถาปัตยกรรมและการปรับปรุงระบบ (System Modernization Roadmap)

ระบบครุภัณฑ์ & บริหารจัดการงานซ่อมบำรุง — โรงพยาบาลพยุหะคีรี

เอกสารนี้วิเคราะห์เจาะลึก 2 ประเด็นหลักตามที่กำหนด พร้อมร่างแนวทางการดำเนินการจริง (Implementation Plan) ในส่วนของการวางระบบ Build & Bundle

🏗️ ส่วนที่ 1: การ Build & Bundle ในระยะยาว (Modern Build Tooling)

1.1 ปัญหาของโครงสร้างเดิม (Current State)

- **Tailwind Play CDN (`cdn.tailwindcss.com`):** โค้ด HTML ปัจจุบันใช้สคริปต์ Play CDN ซึ่งมีขนาด ~3-4 MB และต้องรัน JavaScript เพื่อไล่อ่าน Class ทั้งหมดใน DOM แล้วคอมไพล์ CSS สดบนบราวเซอร์ทุกครั้งที่รีเฟรชหน้าเว็บ ทำให้เกิดอาการหน้าเว็บกระตุก (FOUC: Flash of Unstyled Content) และไม่เหมาะอย่างยิ่งสำหรับการใช้งานจริง (Production)
- **External CDN Dependency:** โมดูลภายนอกทั้งหมด (Chart.js, SheetJS, PptxGenJS, HTML2Canvas, Firebase SDK) โหลดผ่าน Cloudflare/jsDelivr/gstatic หากวันใดที่เครือข่ายโรงพยาบาลมีปัญหา Gateway, หลุด หรือติด Firewall ภายใน จะเปิดระบบไม่ขึ้นทันที
- **สถาปัตยกรรมลูกผสม (Global Scope vs ES Modules):** โค้ดบางส่วนยังผูกตัวแปรไว้กับ `window.*` และมีการแยกโหลดสคริปต์หลายไฟล์ใน ส่งผลให้จัดการ Dependency Order ยาก และเสี่ยงต่อการเกิด Race Condition

1.2 ประโยชน์ที่จะได้รับหลังใช้ Vite & Static Bundle

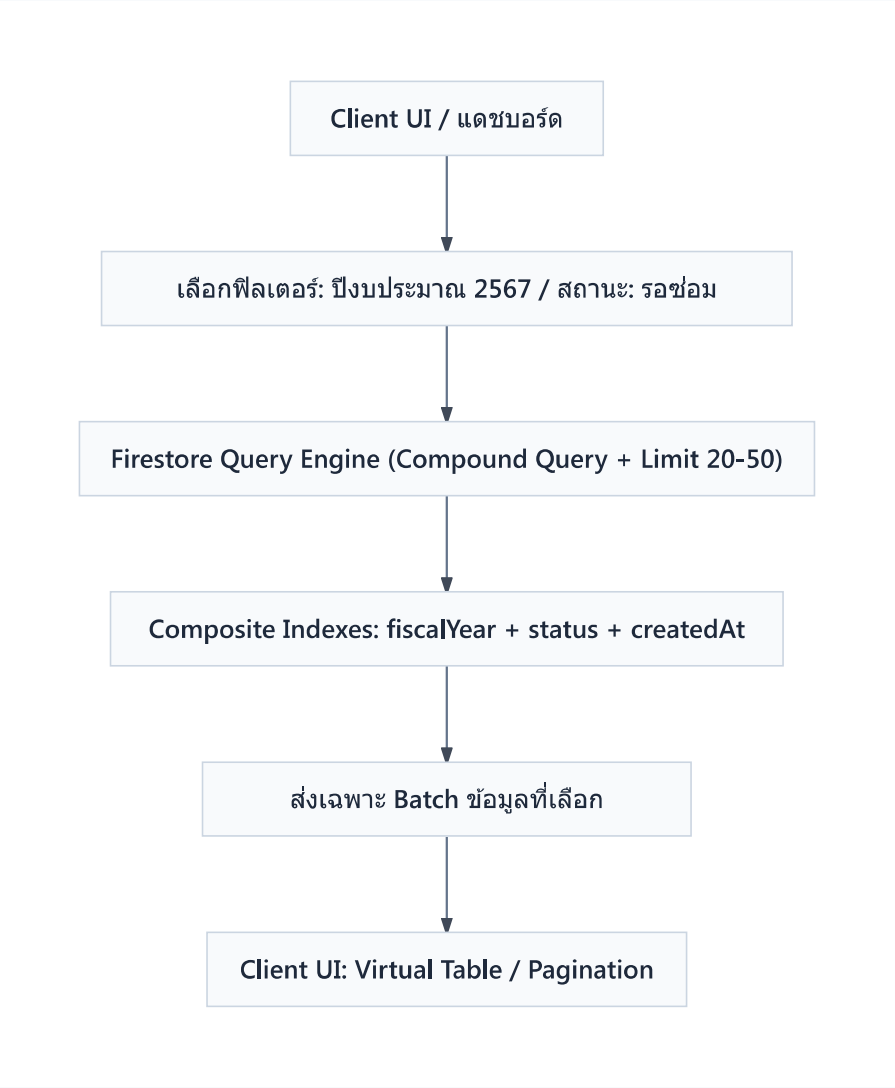
ด้าน	**ก่อนปรับปรุง** (CDN / No-Build)	**หลังปรับปรุง** (Vite + Static Bundle)
ความเร็วในการเปิดหน้าเว็บ	ช้า ต้องดาวน์โหลด CDN หลายตัว และรัน JIT compiler	เร็วขึ้น 3-5 เท่า ไฟล์ CSS ถูกบีบเหลือเฉพาะ Class ที่ใช้จริง (~30-50 KB)
การทำงานใน รพ. (Intranet/Offline)	หากอินเทอร์เน็ตหลุด/บล็อก CDN จะพังทันที	รันได้เสถียรผ่าน Local Server / Intranet ข้อมูล Library อยู่ในเครื่อง
ความปลอดภัยของโค้ด	โค้ด JS กระจัดกระจาย เข้าถึงผ่าน `window` ได้ง่าย	ทำ Tree-shaking, Minification, และ Encapsulation ชัดเจน
Developer Experience	แก้ไขแล้วต้องคอย Ctrl+F5 ป้องกันแคชเก่า	มี Hot Module Replacement (HMR) แก้ไขแล้วหน้าจอเปลี่ยนทันที

⚡ ส่วนที่ 2: การจัดการประสิทธิภาพข้อมูล (Data Querying & Scalability)

2.1 ปัญหาการโหลดข้อมูลเดิม

- ระบบปัจจุบันใช้การ Listen `onSnapshot` หรือโหลดเอกสารทั้งหมดจาก Firestore มาเก็บไว้ใน Array หน่วยความจำของเครื่องไคลเอนต์ (`window.allRepairs` , `window.allEquipments`)
- เมื่อข้อมูลครุภัณฑ์แตะ 5,000 - 20,000 รายการ หรือประวัติซ่อมสะสมหลายปี:
    - **โควตา Firestore พุ่งสูง:** ทุกครั้งที่เปิดหน้าเว็บจะคิดค่า Read ตามจำนวนเอกสารทั้งหมด ทำให้ค่าบริการบานปลาย
    - **Memory Leak บนมือถือ/แท็บเล็ต:** อุปกรณ์พกพาของช่างจะค้างหรือปิดแอพเองจากการโหลด DOM หลายพันรายการพร้อมกัน

2.2 สถาปัตยกรรมการสืบค้นใหม่ (Query Architecture)



1. **Server-Side Filtering ตามปีงบประมาณ & สถานะ:**
   - ใช้ `query(collection(db, "repairs"), where("fiscalYear", "==", 2567), where("status", "==", "pending"), orderBy("createdAt", "desc"), limit(25))`
   - ผู้ใช้จะได้รับเฉพาะข้อมูลที่ต้องการ ไม่ต้องโหลดทั้งฐานข้อมูล
2. **Cursor-based Pagination (Infinite Scroll หรือ Next/Prev):**
   - ใช้ `startAfter(lastVisibleDoc)` สำหรับการดึงข้อมูลหน้าถัดไป ทำให้โหลดทีละ 25-50 รายการได้อย่างรวดเร็วและประหยัด Read
3. **Data Aggregation สำหรับแดชบอร์ด:**
   - สำหรับกราฟสถิติรวม ไม่จำเป็นต้องโหลดทุก Record มานับ แต่ใช้เอกสารสรุปยอดรวม (Summary/Aggregates Document) เช่น `stats/fiscalYear_2567` ที่อัปเดตผ่าน Batch Write หรือ Transaction เมื่อมีการเพิ่ม/แก้ไขงานซ่อม

📋 ส่วนที่ 3: ร่างแบบดำเนินการส่วนที่ 1 (Phase 1 Implementation Plan)

ขั้นตอนที่ 1: การเตรียมสภาพแวดล้อม (Environment Setup)

1. ตรวจสอบและติดตั้ง **Node.js LTS** (v20+ หรือ v22) บนเครื่องพัฒนา
2. สแกนรายการ Library ทั้งหมดที่ใช้อยู่ใน เพื่อเตรียมติดตั้งลงใน `package.json`:
    - `firebase` (^10.x)
    - `chart.js`
    - `xlsx`
    - `jszip`
    - `pptxgenjs`
    - `html2canvas`
    - `tailwindcss`, `postcss`, `autoprefixer`

ขั้นตอนที่ 2: โครงสร้างโฟลเดอร์สำหรับ Vite Project

โครงสร้างจะปรับให้เป็นมาตรฐาน Modern Web App:

```
repairreport-main/
├── dist/                    # โฟลเดอร์ผลลัพธ์พร้อมใช้งาน (Static HTML/CSS/JS รวมสมบูรณ์)
├── src/                     # ซอร์สโค้ดหลัก
│   ├── assets/              # รูปภาพ, ฟอนต์ Sarabun, Prompt แบบ Local
│   ├── styles/
│   │   └── main.css         # Tailwind Directives (@tailwind base; components;
utilities;)
│   ├── js/
│   │   ├── common/          # Unified Firebase, Config, Utils, Audit Log
│   │   ├── repair/          # Core, Dashboard, Staff, Storage, Excel
│   │   ├── parcel/          # Core, Excel, Calculator
│   │   └── main.js          # Entry point รวมการ Import สคริปต์
├── index.html               # HTML หลักที่เรียกใช้งาน /src/main.js
├── tailwind.config.js       # กำหนด Palette สี teal, slate, ฟอนต์ Prompt/Sarabun
├── vite.config.js           # กำหนดการ Bundle และ Base Path
└── package.json
```

ขั้นตอนที่ 3: การแปลง Global Script สู่ Module & Main Entry Point

- จัดการให้ฟังก์ชันที่ HTML ต้องเรียกใช้ผ่าน `onclick="..."` ถูกผูกอย่างถูกต้องผ่าน ES Module Export หรือ Binding อย่างมีระบบ

- ย้าย Tailwind CDN ออก แล้วทดแทนด้วยการคอมไพล์ผ่าน PostCSS/Tailwind CLI ซึ่งจะตัด CSS ที่ไม่ได้ใช้งานออก เหลือเพียงไฟล์ CSS ขนาดเล็กมากๆ

ขั้นตอนที่ 4: การทดสอบและการส่งมอบ (Verification & Build)

1. รันคำสั่ง `npm run dev` เพื่อพัฒนาแบบเรียลไทม์ (Hot Reload)
2. รันคำสั่ง `npm run build` เพื่อตรวจสอบว่าไม่มี Syntax/Type Error และได้โฟลเดอร์ `dist/`
3. ทดสอบการเปิดใช้งานแบบ Offline / Local Web Server ตรวจสอบว่าระบบทำงานครบถ้วนทุกฟังก์ชัน (ส่งซ่อม, คำนวณค่าเสื่อม, Export Excel/PPTX, ดึงข้อมูล Firestore)